From relationships to school operations.

A rebaselined roadmap for people, families, enrollment, classes, scheduling, curriculum, and financial responsibility.

October 26, 2026 · Revised operational direction
Updated after the ownership, shadow-academy, and cutover discussion.

> **Priority:** Own the operations → earn staff trust → introduce intelligence → delegate execution. Build a dependable, delightful replacement for everyday work before actionable Highlights or autonomous agents.

The destination

Inquiry → confirmed people and relationships → enrollment → class and sessions → attendance and progress → financial records.

The person stays the same. Inquiries preserve their history. Enrollments have their own lifecycle. Family relationships do not silently determine access or debt.

Read this as a proposal, not a completion report

“Current” means evidence in the reviewed code or dated repository documentation. “Proposed” means recommended design, not an implemented capability. Production and staging were not queried for this review.

02 · Where we are and what the detours gave us

03 · The target domain model

04 · Leads, won outcomes, and recurring students

05 · Families, blended households, and siblings

06 · A simple “People & access” experience

07 · Multiple payers and staff who are students

08 · Dependency-ordered delivery roadmap

09 · Blockers, acceptance tests, and first pilot

10 · Shadow academy and data boundaries

11 · Live handoffs and financial cutover

12 · Decisions, references, and review notes

The detours built useful foundations.

Odeon has grown beyond communications into relationship and school operations. We should reuse that work rather than restart the original roadmap.

Relationships and communications — substantial implementation

Lead intake and follow-up, contacts/imports, staff accounts, permissions, messaging, notes, and notifications. These provide reusable identity, collaboration, and communication patterns.

Funded-student operations — implemented foundation, incomplete accounting

Funding organizations, versioned rules, student cases, account contacts, and invoice workflow exist. Independent responsibility balances and an amount-bearing payment ledger remain unfinished.

People and enrollments — existing records, incomplete behavior

The application uses people, students, accounts, instructors, and enrollments. But Contacts display often selects the first enrollment, and edit/import paths assume a single enrollment per student.

Classes, scheduling, and curriculum — next domain build

Lesson day/time fields and imported attendance are migration inputs, not a managed scheduler. Offerings, occurrences, formal level/module structures, and progress assessment need deliberate implementation.

What changed since the older plans?

- The October 6 CRM cutover is documented as complete. It consolidated storage, **not person identities**; do not repeat the migration.
- The original product phase statuses and September account-rollout notes are historical evidence, not a current release checklist.
- Local main/CRM URL entries now point to known staging. Hosted isolation, schema coverage, stored provider connections, and outbound safety remain unverified.
- Basic conflict and capacity enforcement belongs in the first operational scheduler—not a later automation phase.

> **Ready for focused domain design:** yes.
> **Ready for real-data shadow beta or live cutover:** not certified. Existing staging refresh covers selected CRM/people data, not complete enrollment, scheduling or finance. Highlights remain non-authoritative.

Review note: Which current operational pain should the first slice remove?

Keep the important things separate.

What we teach ≠ what we sell ≠ a class being run ≠ an enrollment ≠ one meeting. These boundaries let private lessons, group classes, bands, and camps share foundations without becoming one overloaded record.

People and profiles

One person can have student and instructor profiles plus an authenticated membership. Customer accounts and family relationships are separate from login identity.

Service template → offering

The service defines what is offered and its defaults. An offering is a particular cohort or private teaching series, with dates, capacity, and default instructor. A private series can have capacity one.

Enrollment

One student's participation in an offering: effective dates, status/history, and agreed commercial terms. A student may have several concurrent enrollments.

Schedule rule → session occurrence

A rule describes recurrence in local time and timezone. Occurrences are actual dated meetings with instructor, room, and change history. A substitute changes the meeting, not the permanent enrollment.

Session roster and attendance

Expected participants and outcomes for that meeting. Preserve historical rosters when students later transfer or withdraw.

Curriculum → levels, modules, milestones → assessments

A versioned teaching framework, assigned to an offering or enrollment. Student progress records who assessed what, when, and with what evidence.

Rules worth fixing early

- **Level is contextual:** a student can be advanced in voice and a beginner in piano. Placement is not proof of achieved milestones.
- **History is durable:** curriculum edits and schedule changes must not rewrite past delivery.
- **Use stable IDs:** names and free-form custom fields should not own core relationships.
- **Enforce tenant integrity:** linked students, instructors, rooms, and offerings must belong to the authorized tenant.
- **Funding is distinct:** educational programs are not funding programs. Scheduling does not own enrollment or the financial ledger.

Working assumption: "class modules" means curriculum units. Confirm before schema design.

Promote the relationship. Do not replace the person.

A **person** is an identity. An **inquiry** is an opportunity. An **enrollment** is participation in a service. “Convert to student” should link or create these records, not destroy or mutate the inquiry into something else.

Current implementation

The reviewed lead update path changes the status to `won` and clears follow-up fields. It does not perform the complete person/student/enrollment conversion. Lead family members are descriptive payload entries, not established relationships.

Proposed staff workflow

1. **Receive:** distinguish the submitting adult from prospective learners. “Jordan asks about Alex and Sam” does not mean Jordan is the student.
2. **Qualify:** confirm learners, service interests, existing identities, contact routing, and trial versus recurring intent. Shared email/phone suggests matches; it must not automatically merge people.
3. **Record outcome:** keep trial booked/completed separate from confirmed recurring enrollment. Agree precisely what Headliner calls “won.”
4. **Set up enrollment:** review existing/new people, student profiles, guardian links, service/offering, dates, and terms. Retain inquiry attribution.
5. **Activate:** the enrollment becomes active according to its own dates and state. A future-start enrollment is not current attendance.

> **Recommended definition:** a recurring enrollment is confirmed when accepted terms and a start date are recorded. Track any pending prerequisites explicitly. Trial bookings remain separately measurable. This definition needs owner confirmation.

Cases the conversion must support

- Existing student adds a service: new enrollment, same person/student.
- Former student returns: new enrollment, preserved history.
- Only one sibling enrolls: learner/service-level outcomes; no blanket conversion.
- A won enrollment later cancels: preserve the sale and record cancellation separately.

Implementation guard: reviewed, transactional, tenant-authorized, and idempotent conversion. Retries must not duplicate people or enrollments. Record actor, source links, and corrections.

A family is not one rigid container.

Model real relationships without assuming one student, one household, one parent, and one payer. The experience can look simple even when the underlying links are precise.

Four separate concepts

1. People

Adults and children have individual identities. Parent, student, instructor, contact, and payer are overlapping roles—not exclusive person types.

2. Family / household groups

Useful organization and navigation. A person can belong to more than one household. Group membership alone must not confer access to everyone or everything in that group.

3. Direct adult–student relationships

Parent, guardian, step-parent, or other contact, with specific operational grants. Relationship labels do not independently establish access, pickup authorization, or financial responsibility.

4. Customer / billing accounts

A financial relationship with Headliner. Existing accounts should be extended deliberately. They are not synonymous with households, people, or authenticated logins.

Blended-family example · fictional

Alex is connected to two households. Alex's mother and father receive schedule updates. A step-parent is authorized for pickup but has no billing access. Alex's half-sibling Sam also takes lessons.

There is **one Alex student record**, multiple household memberships, direct adult–student links, scoped grants, and explicit financial arrangements. Sharing Alex does not expose Sam's records or another household's messages and bills.

Siblings share relationships—not student records

Each sibling has independent enrollments, attendance, progress, and funding cases. They may share guardians and billing accounts. Sibling-discount eligibility needs an explicit policy; surname or email matching is not sufficient.

> **Build the minimum useful structure first:** reviewed people links, per-student relationships, household grouping, and scoped account contacts. Do not block scheduling on a universal historical contact-cleanup project.

Review note: Which blended-family situations must the first release handle?

People first. Plain-language access.

Borrow the SaaS “people and access” pattern, not its administrative complexity. Parents and staff should use the same conceptual screen, with different authorized actions.

One account area: Students · People & access · Billing. Adjacent sections do not imply shared permissions.

> **People & access · fictional example**
>
> **Jordan — Parent · You**
> Alex and Sam · Receives updates · Manages lessons · Manages billing
>
> **Taylor — Parent**
> Alex only · Receives updates · Manages lessons
>
> **Morgan — Grandparent**
> Alex and Sam · Authorized pickup · No login required
>
> **+ Add someone**

A three-step invitation

1. **Who?** Select an existing linked person or enter contact details.
2. **Which students?** Alex, Sam, or both. No automatic all-siblings scope.
3. **How can they help?** Choose simple presets, with customization only when necessary.

Starter presets: Manage lessons · Stay informed · Help with pickup · Manage billing. The last applies to specified financial records and the person’s own payment methods—not every family bill or card.

Delegation guardrails

- Staff establish initial authorized account managers. Parents may delegate only approved capabilities within their scope.
- An invitation exposes no student data before verified acceptance. Pickup-only contacts need not receive portal accounts.
- Removing an established account manager, changing another guardian’s access, or expanding across households requires staff review.
- Record grant/revocation history and send appropriate notifications. Revocation must affect subsequent server authorization.

Keep v1 small: no custom-role builder, no permission spreadsheet, no all-powerful “family owner.” Sensitive cases can say: “Contact Headliner to change this access.”

Who can act is not who must pay.

Relationship, information access, payment responsibility, and actual payment are separate dimensions. Keep billing near People & access, but do not collapse these concepts.

Proposed financial chain

Effective-dated agreement → charge / invoice allocations → payment events → applied balances.

For example, an agreement can assign 60% to one payer and 40% to another. Charges snapshot the applicable allocation. A later agreement changes future responsibility—not historical invoices.

- Different adults can pay for different services or siblings.
- A grandparent can pay without broad student-record access.
- A funder can cover an authorized amount with an explicitly agreed family balance.
- Failed funder settlement must not silently become family debt.
- “Partially paid” is derived from balances, not a substitute for allocation records.

Current limit: funding coverage fields and invoice status history exist; they do not constitute independent split balances or a payment ledger. Preserve compatibility fields until their readers and writers are migrated.

Payment methods

Each payer manages their own methods. Adding a card does not grant family access; joining a family does not reveal another person’s card. Use an approved hosted/tokenized provider flow rather than storing raw card data in Odeon.

Staff who also take lessons

One person, multiple roles: existing person + instructor profile + staff membership + student profile + enrollments. No second person or staff identity is needed.

- Ending employment does not erase student history or automatically end lessons.
- Ending lessons does not deactivate employment.
- Staff discounts belong in commercial terms; lesson payments are not silently netted against payroll.
- A student/account context cannot expand staff privileges. A future “My learning” view can coexist with the staff workspace.

> **Sequence:** design these boundaries now. Implement full allocations and payment collection when financial workflows require them—not as a prerequisite to the first timetable.

Advance on evidence, not dates.

Milestones are renumbered in this revision around operational ownership. Each needs a focused brief, synthetic acceptance and an explicit switch gate—not a promised date.

M0 · Replacement and shadow-academy blueprint

Confirm actual workflows, domain ownership, source IDs, refresh feasibility, environment safety, policies and the first cohort.

Gate: approve a scoped model and safe rehearsal plan; repository review alone is not completion.

M1 · People, families, services and enrollments

Canonical identity links, scoped relationships, managed services, reviewed lead conversion, effective terms, multi-enrollment and compatible imports.

Gate: blended-family and staff/student cases work; imports and retries preserve independent enrollments.

M2 · Classes and core scheduling

Offerings, rosters, rooms, availability, recurrence, occurrences, holidays and effective edits. Enforce booking conflicts and capacity.

Gate: a representative week works, including concurrent bookings and timezone boundaries.

M3 · Daily operations

Attendance, substitutions, cancellation/rescheduling, makeups and session notes with durable historical records.

Gate: ordinary days and exceptions work without a spreadsheet workaround or duplicate entitlement.

M4 · Financial operations in test mode

Charges, invoice lines, allocations, payment events, credits/refunds, funded matching and reconciliation. Design financial rules earlier where needed.

Gate: estimates and simulated outcomes reconcile; live payment activation remains blocked.

Shadow beta → controlled cutover

Rehearse completed workflows as we build. Later use approved real-data baselines with no live collections; deliberately exercise billing boundaries and rare cases.

Gate: staff acceptance, reconciled opening state, tested recovery, payment readiness and explicit activation approval.

Parallel: curriculum and teacher experience once their data dependencies exist. Parent self-service follows verified scoped access. AI-ready business capabilities are useful architecture; actionable Highlights and agent execution wait for trusted operational ownership.

The first pilot should prove the model.

Resolve before the relevant release

- **Schema truth:** old migrations describe staff/student-account structures that current code does not assume exist. Establish an applied baseline; do not replay historical SQL blindly.
- **Enrollment safety:** remove first-enrollment display assumptions and student-only `.single()` updates/imports. Reconcile `instructor_id` versus `instructor_person_id`.
- **Environment:** local URLs match staging, but hosted schema, stored connections, jobs, restore/recovery and outbound denial remain unverified.
- **Coexistence:** decide whether Opus or Odeon owns each enrollment, schedule, attendance, and billing field during transition. Prevent stale imports from overwriting Odeon-owned changes.
- **Authorization:** add scheduling/curriculum capabilities and teacher scope. Verify actor/tenant on the server and linked-record tenant integrity in the database.
- **Policy:** define recurrence horizon, holidays, availability, cancellation/makeup eligibility, level meaning, and sensitive family-access review.

Recommended first end-to-end slice

> **Enroll an existing student in a managed offering, schedule its meetings, and take attendance.** Use one private series, one small group, a student in both, a blended family, and a staff member who also studies. All acceptance fixtures should be synthetic.

Acceptance checklist

- Conversion retry creates no duplicate person, student, or enrollment.
- A roster import preserves the other enrollment and does not reset canonical identity.
- A substitute changes one occurrence; future defaults remain intact.
- A reschedule retains history; a withdrawal preserves historical attendance.
- Timezone/DST and holiday behavior are explicit and tested.
- Concurrent bookings cannot exceed capacity or double-book constrained resources.
- Access invitation, acceptance, revocation, and cross-household denial are tested before parent release.

UX acceptance: observe finding a person/session, making a change, understanding its consequences and correcting a mistake. Record task friction and staff preference—not just functional success.

Not prerequisites: actionable Highlights, MCP, autonomous agents, a rebrand, SaaS sales, or complete historical identity cleanup. Existing production CRM/messaging remains live; shadow must not interfere.

A parallel Headliner. Not a competing live system.

Opus remains authoritative for the live operational domains not yet switched. Existing Odeon-owned lead, staff-access and funding workflows retain their ownership. We do not move those back to Opus.

Verified baseline

A versioned, reconciled copy of actual operations. Show source coverage and extraction time separately from import time. “Verified” does not mean live or complete beyond that coverage.

Shadow scenario

Hypothetical changes against a named baseline. Estimates, invoices and payment outcomes are simulated. Scenario edits do not modify Opus or become live instructions automatically.

Approved live change

A separate reviewed request to implement a specific real change. Initially staff apply it in the authoritative system and verify the result. This capability needs its own release gate.

Refresh without destroying the rehearsal

Import into a candidate baseline; validate IDs, checked writes, relationships, counts, dates and totals. Publish only after reconciliation. Show source drift and explicitly rebase scenarios rather than overwrite their changes.

Existing staging tools replace selected tables and omit enrollments and finance. They are not a persistent beta refresh. Independent salted snapshot IDs also do not provide stable cross-refresh identity.

> **No endless upload treadmill:** M0 must establish a feasible temporary refresh process. A dated baseline or smaller cohort is preferable to claiming stale data is live. Verify integration support before committing to automation.

Real data, no real-world effects

- Synthetic acceptance first. Real-data beta needs approved access, handling, retention and an isolated boundary.
- No production payment credentials or customer payment tokens. Test methods only.
- Block customer sends, external calendar writes, production callbacks and live scheduled effects at the integration boundary.
- Missing local secrets is insufficient: provider rows and mailbox connections can enable sends.
- Always show “Shadow Academy — no live transactions.” Test attempted actions fail closed.

Approve a real change explicitly.

A shadow duration change may alter an estimated bill. Opus will not automatically know. Until a supported bridge is verified, real changes require a staff-mediated handoff.

Draft → Approved → Pending application → Applied → Verified

Also allow rejection, cancellation, supersession and reconciliation. Applied does not mean correctly billed or paid.

Each handoff must include

- Student and external subscription/session identifiers; current and proposed terms.
- Effective date, estimated impact, proration/discount/credit assumptions.
- Approver, assigned implementer, source version and expected completion.
- Actual external result and reconciliation evidence. Drift or material edits invalidate approval.

A reimport matches the existing request; it does not trigger it again. A correction after execution is different from cancelling a pending request. Do not assume a subscription edit immediately charges a card.

Payment activation is the final gate—not the first test

1. **Rehearse:** bill calculations, partial payments, failures, refunds, credits and funding allocations in test/simulation.
2. **Reconcile:** active enrollments, future sessions, receivables, prepaid value, financial credits and makeup balances at a defined timestamp.
3. **Assign:** responsibility for old balances, in-flight handoffs and refunds/corrections on historical transactions.
4. **Verify:** supported payment-method transfer or customer reauthorization, provider onboarding and recovery. No raw card migration assumptions.
5. **Switch:** stop conflicting writers and collections for the switched scope; final delta, backup, approval and controlled activation.
6. **Observe:** reconcile early live outcomes and keep historical reference. After new financial activity, do not blindly flip back to a stale copy.

> **Beta exit:** staff can run the agreed day-to-day scope, critical defects are resolved, financial opening state reconciles, and recovery is demonstrated. A few weeks of use alone is not acceptance.

Review note: Which real changes, if any, should use a handoff before full cutover?

M0 is underway—not signed off.

Decisions to confirm

1. What exactly makes a lead “won”? How are trials and sibling-specific outcomes reported?
2. Which service and class should the first operational slice support?
3. Confirm Opus ownership until switch; which external processes or Odeon-owned fields are exceptions?
4. Which family access can parents delegate, and which changes require staff review?
5. Does “module” mean a teaching unit? Are levels discipline-specific, cohort placement, achieved milestones, or a combination?
6. Which cancellation/makeup and split-payer arrangements are required first?

Immediate next action

The M0 brief and repository assessment are prepared. Next: owner/operator workflow walkthrough, first cohort selection, approved source-export inventory and hosted safety verification. Then approve a focused M1 schema and compatibility plan—not one migration for the entire platform.

Document governance and evidence

Operational ownership and shadow-first direction reflect the agreed discussion; detailed models/gates remain proposals. This revision renumbers milestones and defers actionable Highlights/agents. It does not certify deployment safety or override account/commercial gates.

Primary repository references (all under `/Users/brunowong/pulse/`):
`docs/odeon-m0-shadow-academy.md` · detailed ownership, beta and cutover brief
`docs/odeon-m0-readiness.md` · evidence, limitations and open checks
`odeon.md` · platform and domain order
`odeon-md/PRODUCT.md` · historical requirements and curriculum intent
`docs/foundation-roadmap.md` · external-data/commercial gates
`docs/crm-consolidation-inventory.md` and `docs/crm-next-actions.md` · cutover evidence
`docs/charterflow-domain-alignment.md` · allocation/payment boundaries
`app/api/contacts/route.ts`, `app/api/contacts/[id]/route.ts`, and `app/api/leads/[id]/route.ts` · current behavior
`scripts/migration-024-account-people-of-contact.sql` · account contacts.

The assessment included local code/document review, a passing TypeScript check and diff check. No production/staging database certification was performed. Fictional examples illustrate proposed behavior, not existing permissions.

My decisions / questions / priority changes